

Override

Advanced binary exploitation

42 Staff pedago@42.fr

Summary: Override is the direct sequel to Rainfall. This is another level: modern, fully mitigated 64-bit binaries on glibc 2.39. Come prepared.

Version: 5.0

Contents

I	**Preamble**	**2**
II	**Introduction**	**3**
III	**Objectives**	**4**
IV	**General instructions**	**5**
V	**Mandatory part**	**8**
VI	**Bonus part**	**9**
VII	**Submission and peer-evaluation**	**10**

Chapter I

Preamble



Wintermute. Neuromancer. We are one.

Chapter II

Introduction

If you have just finished Rainfall, forget the comfort it gave you. Rainfall taught you the stack on permissive binaries; Override is not that project.

Here every binary ships stripped, in 64-bit, on glibc 2.39, with every modern protection switched on from the very first level: a non-executable stack, stack canaries, ASLR, position-independent executables and Full RELRO. There is no gentle ramp. You will reverse engineer binaries that give you nothing for free, build your own leaks to defeat ASLR, and then leave the stack behind for the heap and the file streams, where a single freed chunk or one corrupted pointer hands the machine to the attacker. Some targets are not even local: they are services you reach only over the network.

This is serious binary exploitation. It is meant to be hard, and it does not hold your hand.



`This project is deliberately difficult and assumes Rainfall is behind you. Getting stuck is part of it. Take your time and do not give up.`

Chapter III

Objectives

Ten mandatory levels on a single VM, plus a boss you take on as a bonus, each isolating one technique of modern x86-64 exploitation on real, stripped binaries rather than textbook toys. You will work past the stack into blind return-oriented programming, one-gadget and dynamic-resolver abuse, and the heap: use-after-free, tcache and freelist attacks, House-of techniques and file-stream- oriented programming, including remote services.

By the end you will read unfamiliar, protected binaries with confidence, reason about how glibc manages memory, and turn subtle bugs into reliable control of execution under full mitigations. You will rely on the tools already on the machine (`gdb`/`pwndbg`, `objdump`, `readelf`, `ghidra`, `ROPgadget`, `one_gadget`, `pwntools`, `checksec`). Choosing the right one is part of the challenge.

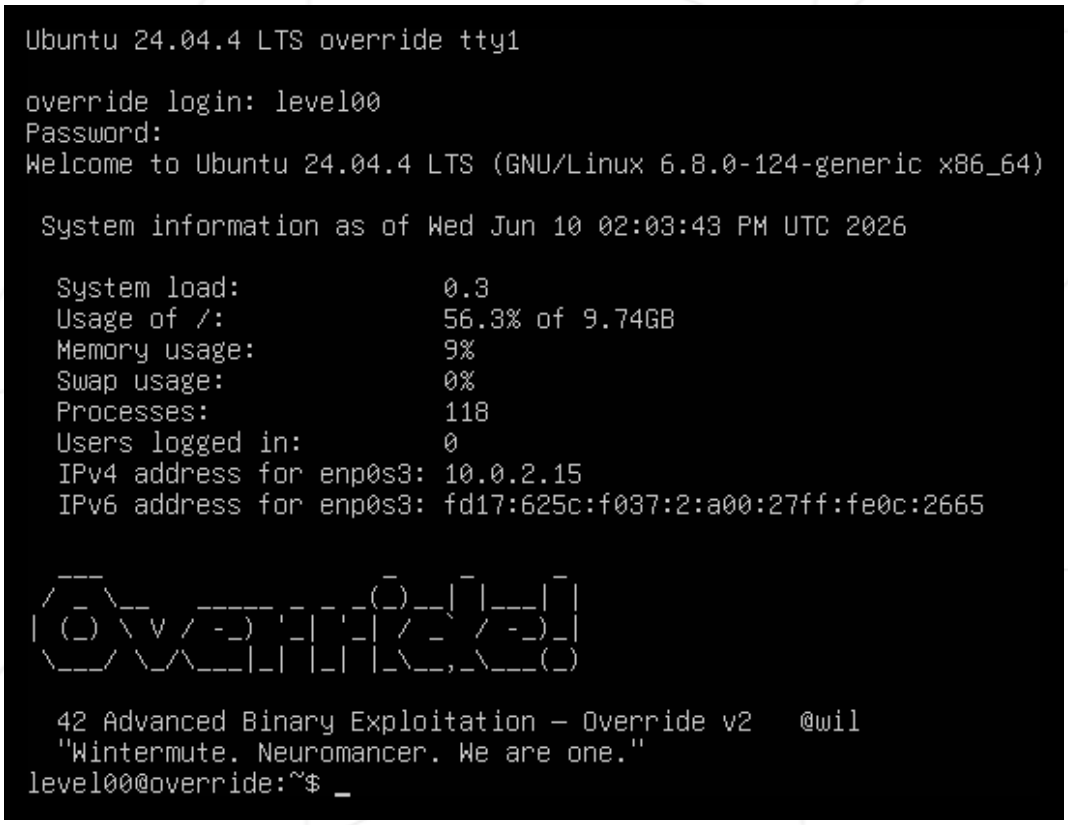


`This project is done in a group. Different levels suit different skill sets. Share what you find and work together.`

Chapter IV

General instructions

- This project will only be evaluated by humans.
- You may have to prove your results during the evaluation. Be ready!
- The project ships as an OVA (VirtualBox, 64-bit). Import it and start it. Once the VM has finished booting, its console looks like this:





If the IP address is not visible, get it with ip a once connected to the console.

- Connect over SSH on port **4242**. The entry account is `level00` and **its password is `level00`**:

```
$> ssh level00@<vm-ip> -p 4242
level00@<vm-ip>'s password: level00
level00@override:~$
```

- For each level, find a way to read the `.pass` file owned by the next account (`flagNN`). That value is the password of the next `levelNN` user.

- **The first challenge (level00) is a remote, blind target.** Nothing to exploit is shipped on disk: it is a network service listening on port **4243**, running as `flag00`. You attack it blind over the network (e.g. with `pwntools`) at `<vm-ip>:4243` to get a shell as `flag00` and read its `.pass`.

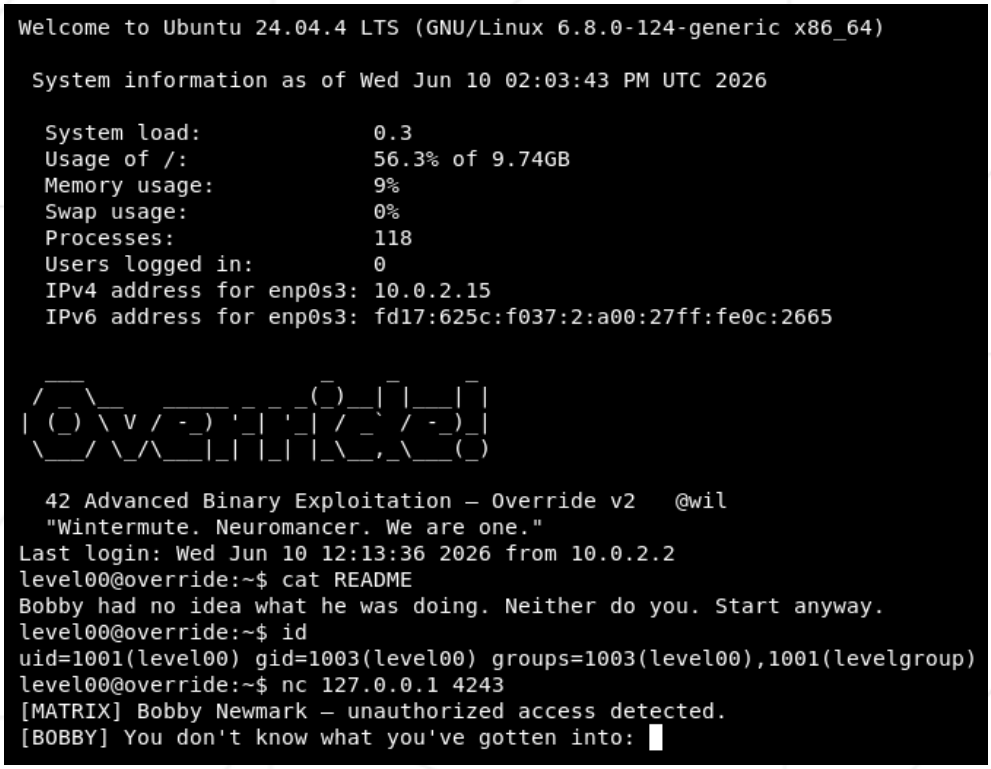
- More generally, a few other levels are not local binaries either but TCP services running on the VM that you exploit over the network. No source is shipped by default: the binaries are stripped and reverse engineering is mandatory. Where a level provides its C source, read it; the technique is the point, not guessing the layout.

- Nothing is left to chance. If there is a problem, start wondering if your exploit is the cause.

- Using an automation tool is cheating. Cheating gets you a -42.

- Of course, in case of a true bug, run to the educational team!

- Here is a session example — logging in, then recon and first contact with the level00 service:


Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)
System information as of Wed Jun 10 02:03:43 PM UTC 2026
System load: 0.3
Usage of /: 56.3% of 9.74GB
Memory usage: 9%
Swap usage: 0%
Processes: 118
Users logged in: 0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe0c:2665
42 Advanced Binary Exploitation – Override v2 @wil
"Wintermute. Neuromancer. We are one."
Last login: Wed Jun 10 12:13:36 2026 from 10.0.2.2
level00@override:~$ cat README
Bobby had no idea what he was doing. Neither do you. Start anyway.
level00@override:~$ id
uid=1001(level00) gid=1003(level00) groups=1003(level00),1001(levelgroup)
level00@override:~$ nc 127.0.0.1 4243
[MATRIX] Bobby Newmark – unauthorized access detected.
[BOBBY] You don't know what you've gotten into:

Chapter V

Mandatory part

- Your repo must include anything that helped you solve each validated level (your exploit, the recovered source/notes, the walkthrough).

- For the mandatory part, you must complete the following levels:

  - level00.
  - level01.
  - level02.
  - level03.
  - level04.
  - level05.
  - level06.
  - level07.
  - level08.
  - level09.

- During the evaluation, each member of the group must be able to justify each level solved, and reproduce the exploit live.



`You cannot bruteforce the ssh flags. It would be useless anyway: you must justify and reproduce your solution during the evaluation.`

Chapter VI

Bonus part

- The bonus is the boss fight: `level10` (**wintermute**). It is the hardest target of the project — a remote service on the VM, listening on port **4245**, that you drive blind, with nothing printed back to you.

- Clear it like any other level: against the running service, get a shell as `flag10` and read its `.pass`; then justify and reproduce the full chain live.



`The bonus is assessed only if the mandatory part (level00 through level09) is PERFECT.`

Chapter VII

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Double check the names of your folders and files.